

Entwicklung eines interaktiven Tutorial- und
Quiz-Systems
für die Lehrlingswerkstatt der Firma Pankl

Diplomarbeit

26. Februar 2026

Inhaltsverzeichnis

Kapitel 1

Einleitung

Die Digitalisierung industrieller Ausbildungsprozesse gewinnt zunehmend an Bedeutung. Unternehmen stehen vor der Herausforderung, theoretisches Wissen und praktische Fähigkeiten effizient zu vermitteln und gleichzeitig moderne Technologien in bestehende Ausbildungsstrukturen zu integrieren. Besonders in technischen Lehrlingswerkstätten ist es erforderlich, Lerninhalte sowohl strukturiert als auch praxisnah aufzubereiten. Klassische Papierunterlagen oder statische Dokumentationen stoßen hierbei an ihre Grenzen, da sie weder interaktive Elemente noch eine adaptive Wissensüberprüfung ermöglichen.

In der Lehrlingswerkstatt der Firma Pankl werden Auszubildende in unterschiedlichen technischen Bereichen geschult. Dabei spielen sowohl theoretische Grundlagen als auch praktische Aufbauten eine zentrale Rolle. Bislang erfolgte die Vermittlung von Arbeitsanleitungen überwiegend in statischer Form. Eine digitale, interaktive Lösung zur strukturierten Darstellung von Tutorials und integrierten Wissensabfragen war nicht vorhanden.

Ziel dieser Diplomarbeit war die Entwicklung eines eigenständigen, interaktiven Tutorial- und Quiz-Systems, das speziell für den Einsatz in der Lehrlingswerkstatt konzipiert wurde. Das System sollte es ermöglichen, schrittweise Anleitungen zum Aufbau technischer Übungen bereitzustellen und ergänzend Quiz-Seiten zur Lernkontrolle einzubinden. Darüber hinaus sollte eine einfache Wartbarkeit und Erweiterbarkeit gewährleistet sein.

Ein wesentliches Entwurfsziel war die Offline-Fähigkeit des Systems. In industriellen Umgebungen kann eine permanente Internetverbindung nicht jederzeit garantiert werden. Das entwickelte System ist daher vollständig offline funktionsfähig. Gleichzeitig wurde eine automatische Update-Funktion integriert, die beim Systemstart eine Verbindung zu einem GitHub-Repository herstellt und neue Versionen des Programmcodes sowie aktualisierte Tutorials bezieht. Dadurch wird eine Kombination aus hoher Betriebssicherheit und zentraler Wartbarkeit erreicht.

Als Hardwareplattform wurde ein Raspberry Pi gewählt. Entscheidende Kriterien waren die kompakte Bauform, der geringe Energieverbrauch, die kostengünstige Beschaffung sowie bereits vorhandene praktische Erfahrung im Umgang mit dieser Plattform aus dem schulischen Unterricht. Der Raspberry Pi eignet sich besonders für Embedded-Systeme und ermöglicht einen zuverlässigen Dauerbetrieb innerhalb eines geschlossenen Schaltkastens.

Die Software wurde in der Programmiersprache Python implementiert. Python zeichnet sich durch eine klare Syntax, hohe Lesbarkeit und schnelle Entwicklungszyklen aus. Für die grafische Benutzeroberfläche wurde das in Python integrierte Framework Tkinter verwendet. Trotz gestalterischer Einschränkungen bietet Tkinter eine stabile und ressourcenschonende Lösung, die für den Betrieb auf einem Raspberry Pi geeignet ist. Die Benutzeroberfläche wurde speziell für einen 10-Point-Touch-Monitor optimiert und verfügt über großflächige Bedienelemente sowie eine klare visuelle Struktur.

Ein weiterer zentraler Bestandteil des Systems ist ein integrierter Administrationsmodus. Dieser kann über einen versteckten Mechanismus aktiviert werden und ist durch ein Passwort geschützt. Dadurch wird verhindert, dass Schüler unbeabsichtigt Tutorials verändern oder löschen. Gleichzeitig ermöglicht der Admin-Modus das Erstellen, Bearbeiten und Verwalten von Lerninhalten direkt am System.

Die vorliegende Arbeit dokumentiert die Konzeption, Implementierung und Evaluierung des entwickelten Systems. Neben technischen Grundlagen werden Architekturentscheidungen, Datenstrukturen, Sicherheitsaspekte sowie Benutzerfreundlichkeit analysiert und reflektiert. Ziel ist es, sowohl die praktische Umsetzbarkeit als auch die didaktische Eignung des Systems für den Einsatz in einer industriellen Ausbildungsumgebung darzustellen.

Kapitel 2

Grundlagen

2.1 Raspberry Pi

2.1.1 Eigenschaften und technische Daten

Der Raspberry Pi ist ein Einplatinencomputer, der speziell für Bildungszwecke und den Einsatz in Embedded-Systemen entwickelt wurde. Die verwendete Version, der Raspberry Pi 4 Model B, verfügt über folgende technische Daten:

- Quad-Core ARM Cortex-A72 Prozessor mit 1,5 GHz
- 4 GB RAM
- MicroSD-Kartensteckplatz für Betriebssystem und Daten
- 2x USB 3.0, 2x USB 2.0 Anschlüsse
- Gigabit Ethernet, WLAN, Bluetooth 5.0
- 1x Micro-HDMI
- GPIO-Pins (40-polig) für Hardware-Erweiterungen
- Kompakte Bauform: ca. 85,6 mm x 56,5 mm
- Geringer Energieverbrauch (ca. 3–7 Watt)

Der Raspberry Pi zeichnet sich durch seine Vielseitigkeit, Erweiterbarkeit und die Unterstützung zahlreicher Betriebssysteme (z.B. Raspberry Pi OS, Linux-Distributionen) aus. Die Hardware ist für den Dauerbetrieb ausgelegt und eignet sich besonders für den Einsatz in industriellen und schulischen Projekten.



Abbildung 2.1: Raspberry Pi 4 Model B – typische Hardwareausstattung

2.1.2 Einsatz im Embedded-Bereich

Der Raspberry Pi wird häufig als Steuerzentrale in Embedded-Systemen eingesetzt. Typische Anwendungen sind Automatisierung, Messdatenerfassung, Steuerung von Maschinen, IoT-Projekte und Prototypenentwicklung. Durch die GPIO-Pins können Sensoren, Aktoren und andere Hardware-Komponenten direkt angebunden werden. Die kompakte Bauform und der geringe Energieverbrauch ermöglichen den Einsatz in geschlossenen Gehäusen und mobilen Systemen.

Beispielhafte Anwendungsfälle im Embedded-Bereich sind:

- Steuerung und Überwachung von Prozessen
- Anbindung von Sensoren und Aktoren über GPIO, I2C, SPI und UART
- Datenlogging und Visualisierung
- Entwicklung von Prototypen und Testsystemen

- Integration in industrielle Anlagen und Lehrmittel

Die breite Unterstützung von Programmiersprachen und Betriebssystemen macht den Raspberry Pi besonders flexibel für Embedded-Anwendungen. Zudem stehen zahlreiche Bibliotheken und Frameworks zur Verfügung, die die Entwicklung und Integration erleichtern.



Abbildung 2.2: Raspberry Pi im Embedded-Einsatz, z.B. im Schaltschrank mit Sensoren

2.1.3 Vorteile

Der Raspberry Pi bietet zahlreiche Vorteile für den Einsatz in technischen und industriellen Projekten:

- Kostengünstige Anschaffung im Vergleich zu klassischen Industrie-PCs
- Kompakte Bauform und geringer Platzbedarf
- Niedriger Energieverbrauch
- Große Community und umfangreiche Dokumentation

- Vielfältige Schnittstellen und Erweiterungsmöglichkeiten
- Unterstützung zahlreicher Betriebssysteme und Programmiersprachen
- Einfache Integration in bestehende Systeme

Diese Vorteile machen den Raspberry Pi besonders attraktiv für Bildungseinrichtungen, Prototypenbau und industrielle Anwendungen.



Abbildung 2.3: Vergleich: Raspberry Pi und klassische Industrie-PCs

2.1.4 Eignung für Dauerbetrieb

Der Raspberry Pi ist für den Dauerbetrieb konzipiert und wird häufig in Anwendungen eingesetzt, die eine hohe Zuverlässigkeit erfordern. Die Hardware ist robust und kann auch unter Dauerlast stabil arbeiten. Durch den geringen Energieverbrauch eignet sich der Raspberry Pi ideal für den Einsatz in geschlossenen Gehäusen und Umgebungen mit begrenzter Stromversorgung.

Wichtige Aspekte für den Dauerbetrieb sind:

- Zuverlässiger Betrieb über Monate oder Jahre
- Geringe Wärmeentwicklung
- Geeignet für den Einsatz in industriellen Steuerungen und Überwachungssystemen
- Einfache Wartung und Updates

Die Erfahrungen aus zahlreichen Projekten zeigen, dass der Raspberry Pi auch im Dauerbetrieb eine hohe Stabilität und Betriebssicherheit bietet.

2.2 Programmiersprache Python

2.2.1 Syntax und Lesbarkeit

Python zeichnet sich durch eine klare und leicht verständliche Syntax aus. Der Code ist gut lesbar und ermöglicht es auch Einsteigern, Programme schnell zu verstehen und zu schreiben. Durch die strikte Einrückung wird die Struktur des Codes zusätzlich hervorgehoben. Kommentare und sprechende Variablennamen werden in der Python-Community besonders gefördert, was die Verständlichkeit weiter erhöht. Dies erleichtert die Zusammenarbeit im Team und die spätere Wartung von Projekten.

2.2.2 Wartbarkeit von Code

Die hohe Lesbarkeit und die modulare Struktur von Python-Projekten erleichtern die Wartung und Erweiterung des Codes. Funktionen und Klassen können übersichtlich organisiert werden, was die Fehlersuche und Weiterentwicklung vereinfacht. Durch die dynamische Typisierung und die Möglichkeit, Code in Modulen und Paketen zu strukturieren, lassen sich auch größere Projekte übersichtlich gestalten. Zudem stehen zahlreiche Tools für automatisierte Tests und Code-Analyse zur Verfügung.

2.2.3 Schnelle Entwicklungszeit

Python ermöglicht durch seine umfangreiche Standardbibliothek und die Vielzahl an verfügbaren Paketen eine sehr schnelle Entwicklung von Anwendungen. Viele Aufgaben können mit wenigen Zeilen Code umgesetzt werden. Die große Auswahl an Bibliotheken für verschiedenste Anwendungsbereiche (z.B. Webentwicklung, Datenanalyse, Machine Learning, Automatisierung) beschleunigt die Entwicklung zusätzlich. Auch die einfache Installation von Paketen über den Python Package Index (PyPI) trägt zur Effizienz bei.

2.2.4 Vorteile im Ausbildungsumfeld

Aufgrund der einfachen Syntax und der großen Community eignet sich Python besonders gut für den Einsatz in der Ausbildung. Lernende können schnell eigene Projekte umsetzen und erhalten Unterstützung durch zahlreiche Online-Ressourcen. Python wird weltweit an Schulen und Universitäten eingesetzt und ist für viele der erste Kontakt mit Programmierung. Die Sprache fördert das algorithmische Denken und ermöglicht es, komplexe Sachverhalte auf verständliche Weise zu vermitteln. Zudem gibt es viele frei verfügbare Lehrmaterialien und interaktive Lernplattformen.

2.3 GUI-Framework Tkinter

2.3.1 Integration in Python

Tkinter ist das Standard-GUI-Toolkit für Python und bereits in der Standardinstallation enthalten. Es ermöglicht die einfache Erstellung grafischer Benutzeroberflächen ohne zusätzliche Abhängigkeiten. Die Programmierung erfolgt vollständig in Python, sodass keine weiteren Programmiersprachen oder Frameworks erlernt werden müssen. Durch die enge Integration in Python lassen sich GUI-Elemente direkt mit der Anwendungslogik verbinden.

2.3.2 Ressourcenverbrauch

Tkinter benötigt im Vergleich zu anderen GUI-Frameworks nur wenig Speicher und Rechenleistung. Dadurch eignet es sich besonders für den Einsatz auf ressourcenschwacher Hardware wie dem Raspberry Pi. Selbst auf älteren oder leistungsschwachen Geräten laufen Tkinter-Anwendungen stabil und flüssig. Dies ist ein entscheidender Vorteil für Embedded-Systeme und Projekte mit begrenzten Ressourcen.

2.3.3 Plattformunabhängigkeit

Anwendungen, die mit Tkinter entwickelt wurden, laufen auf verschiedenen Betriebssystemen wie Windows, Linux und macOS, ohne dass der Quellcode angepasst werden muss. Die grafische Oberfläche passt sich automatisch an das jeweilige Betriebssystem an. Dadurch können Programme einfach verteilt und auf unterschiedlichen Plattformen eingesetzt werden, was die Wartung und Weiterentwicklung erleichtert.

2.3.4 Einschränkungen im Design

Die Gestaltungsmöglichkeiten von Tkinter sind im Vergleich zu modernen Frameworks eingeschränkt. Komplexe oder besonders ansprechende Designs lassen sich nur mit zu-



Abbildung 2.4: Beispiel einer einfachen Tkinter-Benutzeroberfläche

sätzlichem Aufwand umsetzen. Moderne Animationen, ausgefallene Layouts oder spezielle Bedienelemente sind nur eingeschränkt realisierbar. Für viele Standardanwendungen reicht der Funktionsumfang jedoch aus, insbesondere wenn Funktionalität und Stabilität im Vordergrund stehen.

2.3.5 Eignung für Touch-Systeme

Durch die Unterstützung großer Bedienelemente und einfacher Bedienkonzepte eignet sich Tkinter gut für Touch-Anwendungen, wie sie auf dem Raspberry Pi mit Touchscreen zum Einsatz kommen. Buttons und andere Steuerelemente können so gestaltet werden, dass sie auch mit dem Finger zuverlässig bedient werden können. Die einfache Anpassung der Oberflächenelemente ermöglicht eine gute Usability auf Touch-Geräten.

2.4 Datenhaltung mittels JSON

2.4.1 Strukturierte Speicherung

JSON (JavaScript Object Notation) ermöglicht die strukturierte und hierarchische Speicherung von Daten. Komplexe Datenstrukturen wie Listen und Objekte können übersichtlich abgebildet werden. Die klare Struktur erleichtert die Verarbeitung der Daten sowohl durch Menschen als auch durch Maschinen. In Python lassen sich JSON-Daten einfach mit den Standardbibliotheken einlesen und schreiben.

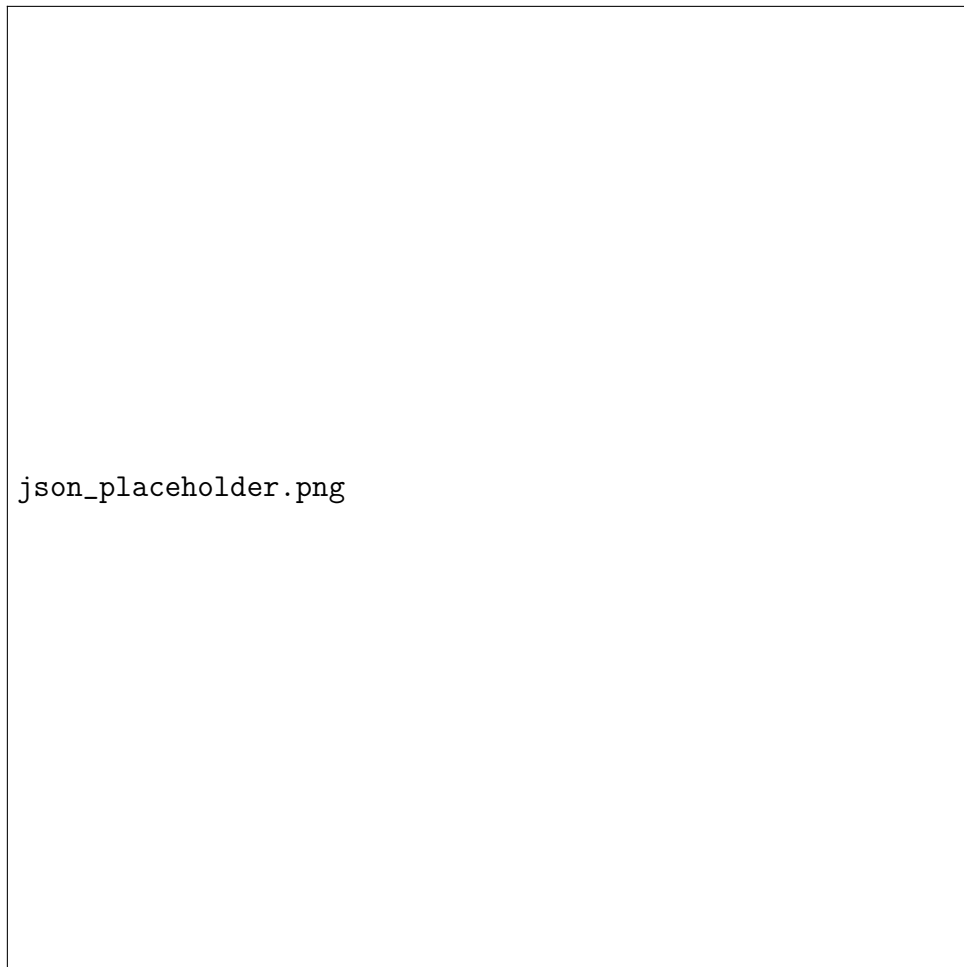


Abbildung 2.5: Ausschnitt einer typischen JSON-Datei zur Datenhaltung

2.4.2 Lesbarkeit

JSON-Dateien sind menschenlesbar und können mit jedem Texteditor betrachtet und bearbeitet werden. Dies erleichtert die Fehlersuche und Anpassung der gespeicherten Daten. Auch für die Dokumentation und den Austausch von Daten zwischen verschiedenen Systemen ist die Lesbarkeit von Vorteil. Fehler in der Datenstruktur lassen sich schnell erkennen und beheben.

2.4.3 Keine externe Datenbank notwendig

Für viele Anwendungen reicht die Speicherung der Daten in JSON-Dateien aus. Es ist keine zusätzliche Datenbanksoftware erforderlich, was die Systemarchitektur vereinfacht. Dies reduziert den Wartungsaufwand und die Komplexität des Systems. Besonders für kleine bis mittlere Projekte ist diese Lösung oft vollkommen ausreichend.

2.4.4 Vorteile für Offline-Systeme

Da JSON-Dateien lokal gespeichert werden, ist der Zugriff auf die Daten auch ohne Internetverbindung möglich. Dies ist besonders für Systeme im Offline-Betrieb von Vorteil. Die Daten sind jederzeit verfügbar und können bei Bedarf einfach gesichert oder übertragen werden. Auch Updates und Anpassungen lassen sich unkompliziert durchführen, ohne dass eine zentrale Datenbank angepasst werden muss.

Kapitel 3

Anforderungsanalyse

3.1 Funktionale Anforderungen

3.1.1 Anzeige von Tutorials

Das System muss in der Lage sein, Tutorials in strukturierter Form darzustellen. Die Tutorials bestehen aus mehreren Seiten, die Schritt für Schritt durchgearbeitet werden können. Jede Seite enthält Text, Bilder und ggf. Quiz-Fragen. Die Navigation zwischen den Seiten erfolgt intuitiv und ist für Touch-Bedienung optimiert.

3.1.2 Unterstützung von Inhalts- und Quiz-Seiten

Neben klassischen Inhaltsseiten, die Wissen vermitteln, gibt es Quiz-Seiten zur Überprüfung des Lernerfolgs. Das System muss beide Seitentypen unterstützen und zwischen ihnen unterscheiden können. Quiz-Seiten enthalten Multiple-Choice-Fragen, deren Beantwortung den Fortschritt im Tutorial beeinflusst.

3.1.3 Bildintegration

Bilder sind ein zentraler Bestandteil der Tutorials und dienen der Veranschaulichung technischer Abläufe. Das System muss die Einbindung und Anzeige von Bildern auf jeder Seite ermöglichen. Die Bilder werden lokal gespeichert und können vom Administrator hinzugefügt oder entfernt werden.

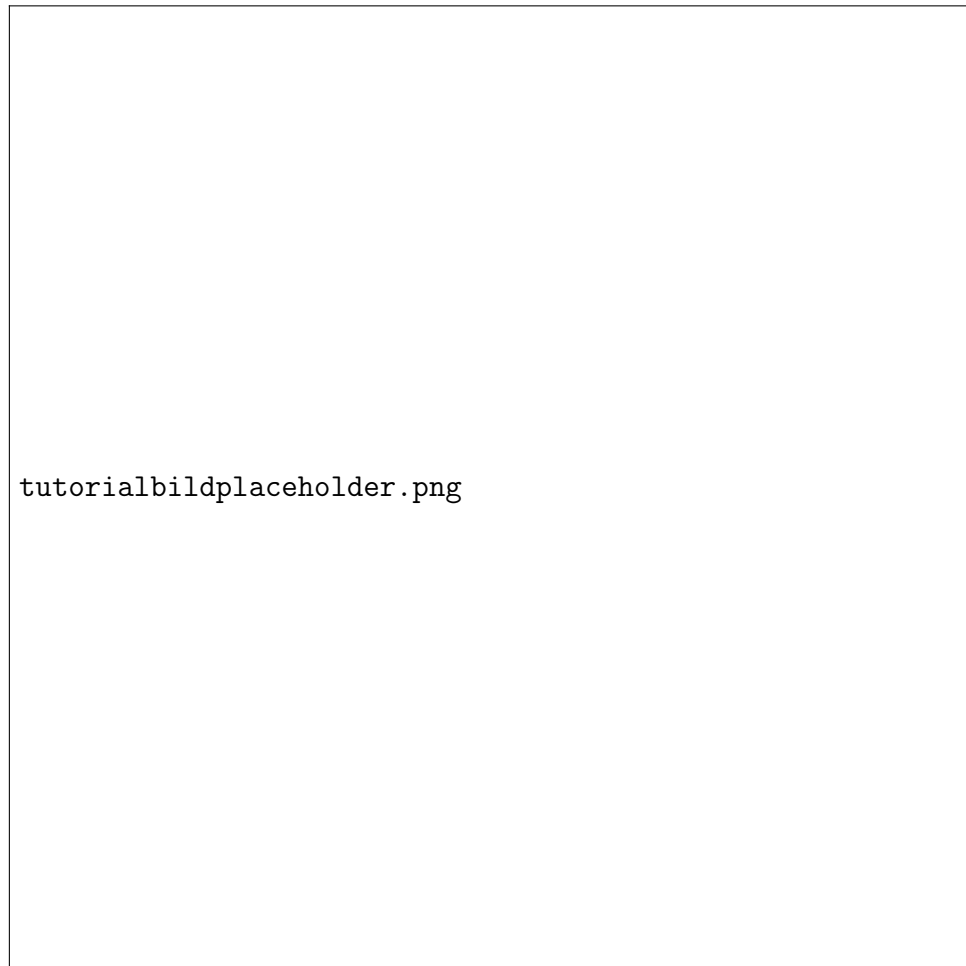


Abbildung 3.1: Screenshot einer Tutorialseite mit integriertem Bild

3.1.4 Zufällige Antwortreihenfolge im Quiz

Um das Auswendiglernen von Antworten zu erschweren, werden die Antwortmöglichkeiten auf Quiz-Seiten bei jedem Durchlauf zufällig angeordnet. Das System sorgt dafür, dass die richtige Antwort immer enthalten ist und die Reihenfolge variiert.

3.1.5 Fortschritt nur bei richtiger Antwort

Der Fortschritt im Tutorial ist an die richtige Beantwortung der Quiz-Fragen gekoppelt. Erst wenn die korrekte Antwort ausgewählt wurde, kann zur nächsten Seite gewechselt werden. Dies stellt sicher, dass das Wissen tatsächlich überprüft wird.

3.1.6 Admin-Modus mit Passwortschutz

Der Administrationsmodus ermöglicht das Erstellen, Bearbeiten und Löschen von Tutorials und Quiz-Fragen. Der Zugang zum Admin-Modus ist durch ein Passwort geschützt, um unbefugte Änderungen durch Benutzer zu verhindern. Der Admin-Modus ist über einen versteckten Mechanismus erreichbar und bietet erweiterte Funktionen zur Verwaltung der Lerninhalte.



Abbildung 3.2: Admin-Modus: Passwortabfrage oder Editor-Ansicht

3.1.7 Erstellung, Bearbeitung und Löschung von Tutorials

Das System bietet einen Editor, mit dem Tutorials und Quiz-Seiten direkt am Gerät erstellt, bearbeitet und gelöscht werden können. Änderungen werden sofort gespeichert und stehen den Nutzern zur Verfügung.



Abbildung 3.3: Editor zur Erstellung und Bearbeitung von Tutorials

3.1.8 Offline-Betrieb

Das System ist vollständig offline-fähig und benötigt keine permanente Internetverbindung. Alle Daten und Tutorials werden lokal gespeichert und sind jederzeit verfügbar.



Abbildung 3.4: Symbolbild: Offline-Betrieb des Systems

3.1.9 Automatisches Update über GitHub beim Systemstart

Beim Systemstart wird automatisch geprüft, ob eine neue Version des Programms oder der Tutorials im GitHub-Repository verfügbar ist. Updates werden heruntergeladen und installiert, sofern eine Internetverbindung besteht.



Abbildung 3.5: Symbolbild: Automatisches Update über GitHub

3.2 Nicht-funktionale Anforderungen

3.2.1 Touch-Optimierung

Die Benutzeroberfläche ist speziell für Touch-Bedienung ausgelegt. Große Buttons und übersichtliche Layouts sorgen für eine einfache Bedienung.

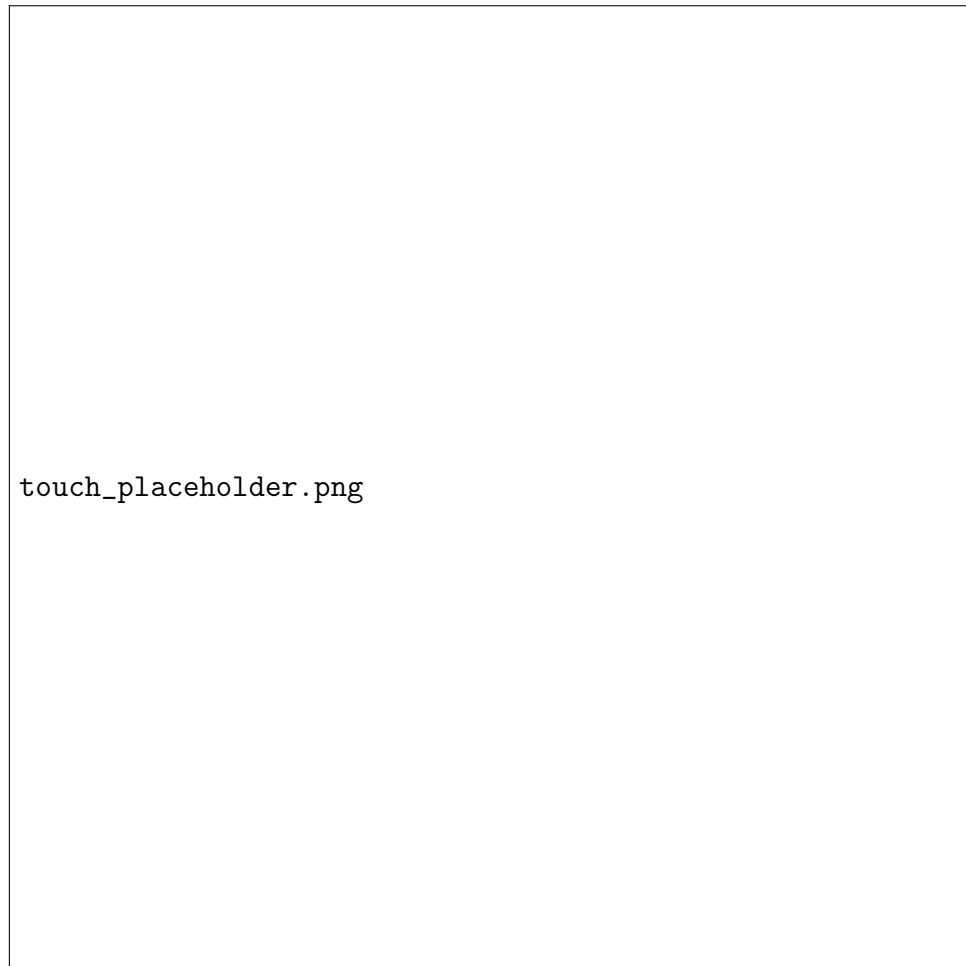


Abbildung 3.6: Touch-optimierte Benutzeroberfläche

3.2.2 Übersichtliche Benutzeroberfläche

Die Benutzeroberfläche ist klar strukturiert und ermöglicht eine intuitive Navigation. Alle wichtigen Funktionen sind schnell erreichbar und die Darstellung ist auf das Wesentliche reduziert.



Abbildung 3.7: Übersichtliche und klar strukturierte Benutzeroberfläche

3.2.3 Wartbarkeit des Codes

Der Quellcode ist modular aufgebaut und gut dokumentiert. Änderungen und Erweiterungen können einfach vorgenommen werden. Die Verwendung von Python und klaren Strukturen erleichtert die Wartung.

3.2.4 Stabilität im Dauerbetrieb

Das System ist für den Dauerbetrieb ausgelegt und wurde auf dem Raspberry Pi ausgiebig getestet. Auch bei längerer Nutzung bleibt die Performance stabil und es treten keine Speicherprobleme auf.



Abbildung 3.8: Langzeitbetrieb: Stabilität und Zuverlässigkeit des Systems

3.2.5 Klare Trennung von Benutzer- und Administrationsrechten

Es gibt eine klare Trennung zwischen normalen Benutzern und Administratoren. Nur Administratoren können Tutorials bearbeiten oder löschen, während Benutzer ausschließlich Inhalte konsumieren und Quizfragen beantworten können.

Kapitel 4

Systemarchitektur

4.1 Gesamtstruktur

4.1.1 `main.py` als Einstiegspunkt

Die Datei `main.py` dient als Einstiegspunkt für das gesamte System. Hier wird die Anwendung gestartet und die grundlegende Initialisierung durchgeführt.

4.1.2 app_core.py als zentrale Steuerung

Das Modul `app_core.py` übernimmt die zentrale Steuerung der Anwendung. Es verwaltet die Navigation zwischen den verschiedenen Screens und steuert die Kernlogik.



Abbildung 4.1: Flussdiagramm: Steuerung der Anwendung durch `app_core.py`

4.1.3 Screen-Module (Home, Editor, Viewer, Settings)

Die verschiedenen Screen-Module sind für die Darstellung und Interaktion der jeweiligen Ansichten zuständig. Dazu gehören der HomeScreen, der Editor, der Viewer und die Einstellungen.



Abbildung 4.2: Screens: Home, Editor, Viewer und Settings

4.1.4 `data_store` für Dateiverwaltung

Das Modul `data_store` übernimmt die Verwaltung und Speicherung der Tutorialdaten und Bilder. Es sorgt für das Laden und Speichern der JSON-Dateien.



Abbildung 4.3: Symbolbild: Verwaltung der Daten durch `data_store`

4.1.5 `image_utils` für Bildverarbeitung

Das Modul `image_utils` bietet Funktionen zur Verarbeitung und Anzeige von Bildern. Dazu gehören das Skalieren, Zuschneiden und Konvertieren von Bilddateien.

4.2 Ordnerstruktur

4.2.1 `tut_data/`

Der Ordner `tut_data/` enthält alle Tutorialdaten und Bilder. Hier werden die einzelnen Tutorials und deren zugehörige Ressourcen abgelegt.

4.2.2 tutorials/

Im Ordner `tutorials/` werden die einzelnen Tutorials als JSON-Dateien gespeichert. Jede Datei repräsentiert ein Tutorial mit mehreren Seiten.

4.2.3 quizzes/

Der Ordner `quizzes/` enthält die Quizdaten, die den Tutorials zugeordnet sind. Die Quizfragen werden ebenfalls als JSON-Dateien abgelegt.

4.2.4 gemischt/

Im Ordner `gemischt/` werden verschiedene Tutorials und Quizdaten zusammengeführt. Dies ermöglicht eine flexible Zuordnung und Mischung von Inhalten.

4.2.5 Jedes Tutorial mit eigener data.json

Für jedes Tutorial existiert eine eigene `data.json`-Datei, in der alle relevanten Informationen und Seiten gespeichert sind.

4.2.6 Separater Image-Ordner pro Tutorial

Zu jedem Tutorial gibt es einen separaten Image-Ordner, in dem die zugehörigen Bilder abgelegt werden. Dies sorgt für Übersichtlichkeit und eine einfache Verwaltung.

4.3 Datenmodell

4.3.1 ID-basierte Identifikation



Abbildung 4.4: Beispiel: ID-basierte Zuordnung von Tutorials und Seiten

Tutorials und Quizseiten werden über eindeutige IDs identifiziert. Dies erleichtert die Zuordnung und Verwaltung der Inhalte.

4.3.2 Seitentypen: content, quiz

Es gibt zwei Seitentypen: Inhaltsseiten (content) und Quizseiten (quiz). Inhaltsseiten vermitteln Wissen, Quizseiten überprüfen das Verständnis.

4.3.3 JSON-Struktur

Die Daten werden in einer klar definierten JSON-Struktur gespeichert. Diese Struktur ermöglicht eine einfache Verarbeitung und Erweiterung.



Abbildung 4.5: Beispielhafte JSON-Struktur für Tutorials und Quizdaten

4.3.4 Trennung von Metadaten und Seiteninhalt

Metadaten (z.B. Titel, Autor, Datum) werden von den eigentlichen Seiteninhalten getrennt gespeichert. Dies sorgt für Übersichtlichkeit und erleichtert die Verwaltung.

Kapitel 5

Implementierung

5.1 Benutzeroberfläche

5.1.1 Kartenlayout im HomeScreen

Der HomeScreen verwendet ein Kartenlayout, um die Tutorials übersichtlich darzustellen. Jede Karte repräsentiert ein Tutorial und zeigt wichtige Informationen auf einen Blick.

5.1.2 Vorschau-Icons

Tutorials werden im HomeScreen mit Vorschau-Icons dargestellt, die einen schnellen Überblick über den Inhalt bieten.

5.1.3 Scrollbare Oberfläche

Die Oberfläche ist scrollbar gestaltet, sodass auch bei vielen Tutorials und Seiten eine einfache Navigation möglich ist.

5.1.4 Touch-optimierte Buttons

Alle Buttons sind für die Touch-Bedienung optimiert und lassen sich auch mit dem Finger zuverlässig bedienen.

5.2 Quiz-System

5.2.1 Dynamische Antwortbuttons

Im Quiz-System werden die Antwortbuttons dynamisch generiert. Die Reihenfolge und Anzahl der Antworten kann flexibel angepasst werden.



karten_placeholder.png

Abbildung 5.1: Kartenlayout im HomeScreen zur Darstellung der Tutorials

5.2.2 Zufällige Reihenfolge

Die Reihenfolge der Antwortmöglichkeiten wird bei jedem Durchlauf zufällig gewählt, um das Auswendiglernen zu erschweren.

5.2.3 Richtige Antwort erforderlich für Fortschritt

Nur bei Auswahl der richtigen Antwort kann im Quiz-System zum nächsten Schritt gewechselt werden. Dies stellt sicher, dass das Wissen tatsächlich überprüft wird.

5.2.4 Feedbackmechanismus

Nach der Beantwortung einer Quizfrage erhält der Nutzer direktes Feedback, ob die Antwort richtig oder falsch war.

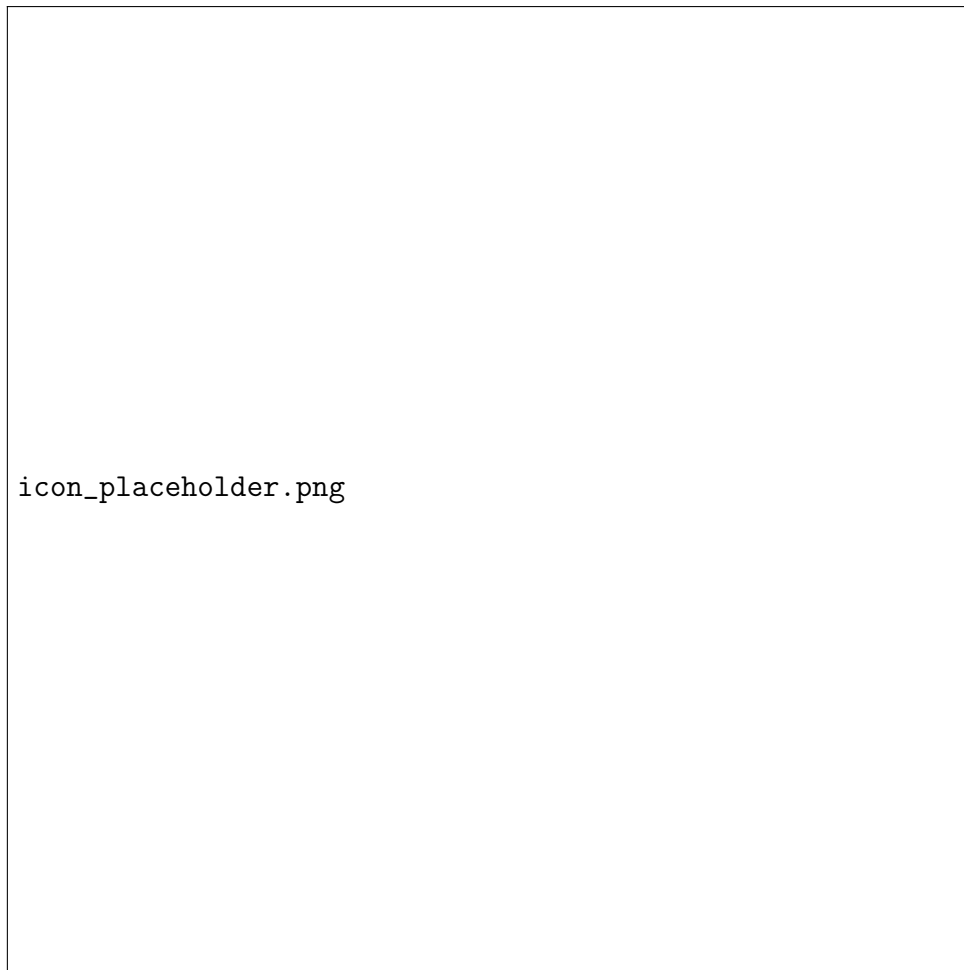


Abbildung 5.2: Vorschau-Icon im HomeScreen

5.3 Editor

5.3.1 Dynamisches Hinzufügen von Seiten

Im Editor können neue Seiten dynamisch hinzugefügt werden. Dies ermöglicht eine flexible Erweiterung der Tutorials.

5.3.2 Löschen einzelner Seiten

Seiten können im Editor einzeln gelöscht werden, ohne das gesamte Tutorial zu entfernen.

5.3.3 Bildintegration

Bilder können im Editor einfach zu den einzelnen Seiten hinzugefügt werden. Die Integration erfolgt über den Image-Ordner des jeweiligen Tutorials.



Abbildung 5.3: Scrollbare Oberfläche für viele Tutorials und Seiten

5.3.4 Trennung zwischen Inhalts- und Quiz-Seiten

Im Editor wird klar zwischen Inhaltsseiten und Quizseiten unterschieden. Dies erleichtert die Strukturierung und Verwaltung der Tutorials.

5.4 Admin-Modus

5.4.1 Versteckter Aktivierungsmechanismus

Der Admin-Modus kann nur über einen versteckten Mechanismus aktiviert werden, um unbefugten Zugriff zu verhindern.

5.4.2 Passwortabfrage

Beim Wechsel in den Admin-Modus wird eine Passwortabfrage eingeblendet. Nur berechnigte Personen können Änderungen vornehmen.



Abbildung 5.4: Direktes Feedback nach Beantwortung einer Quizfrage

5.4.3 Rechteverwaltung

Die Rechteverwaltung sorgt dafür, dass nur Administratoren Tutorials bearbeiten oder löschen können. Benutzer haben ausschließlich Leserechte.

Kapitel 6

Tests und Validierung

6.1 Funktionstests

6.2 Belastungstest auf Raspberry Pi

6.3 Touch-Bedienbarkeit

6.4 Fehlerszenarien

6.5 Benutzerakzeptanz in der Lehrlingswerkstatt

Kapitel 7

Probleme und Lösungen

7.1 Limitierungen von Tkinter

7.2 Scrollbare Frames

7.3 Bildspeicherverwaltung

7.4 UI-Layout-Probleme

7.5 Rechteverwaltungskonzept

Kapitel 8

Erweiterungsmöglichkeiten

8.1 Datenbankbindung

8.2 Online-Synchronisation

8.3 Benutzerkonten

8.4 Statistikfunktionen

8.5 Mehrsprachigkeit

Kapitel 9

Fazit

9.1 Zielerreichung

9.2 Bewertung der technischen Umsetzung

9.3 Praxistauglichkeit

9.4 Persönliche Lernerfahrungen